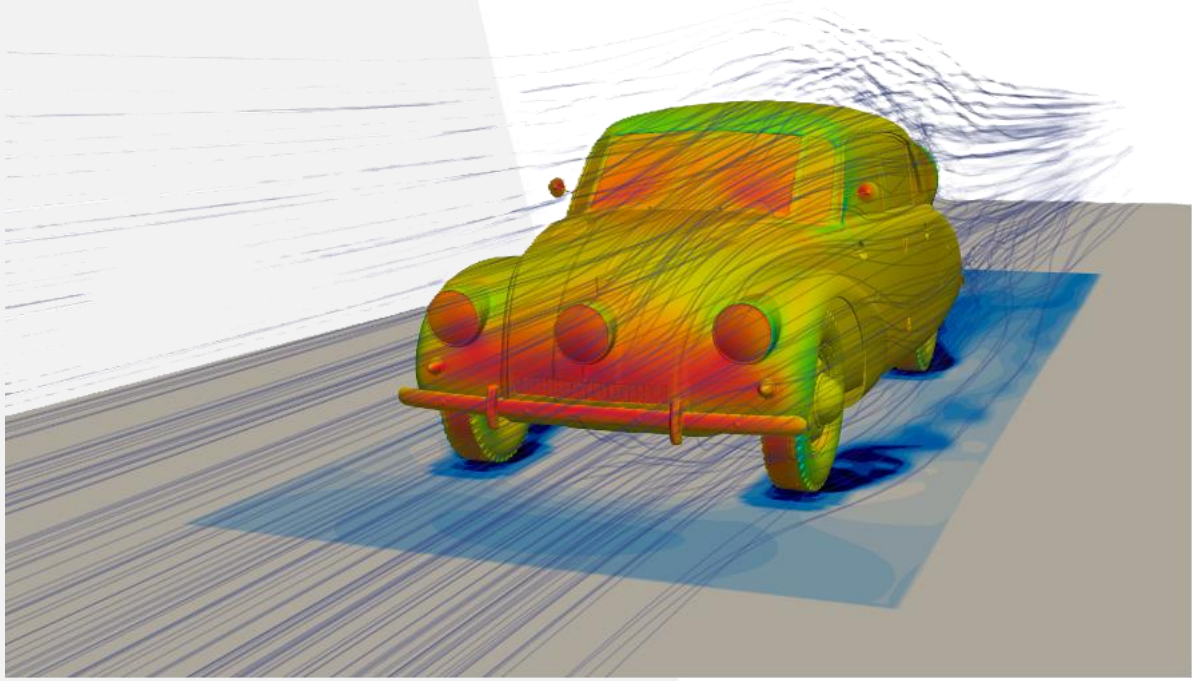
KEYSIGHT



OpenFOAM HPC

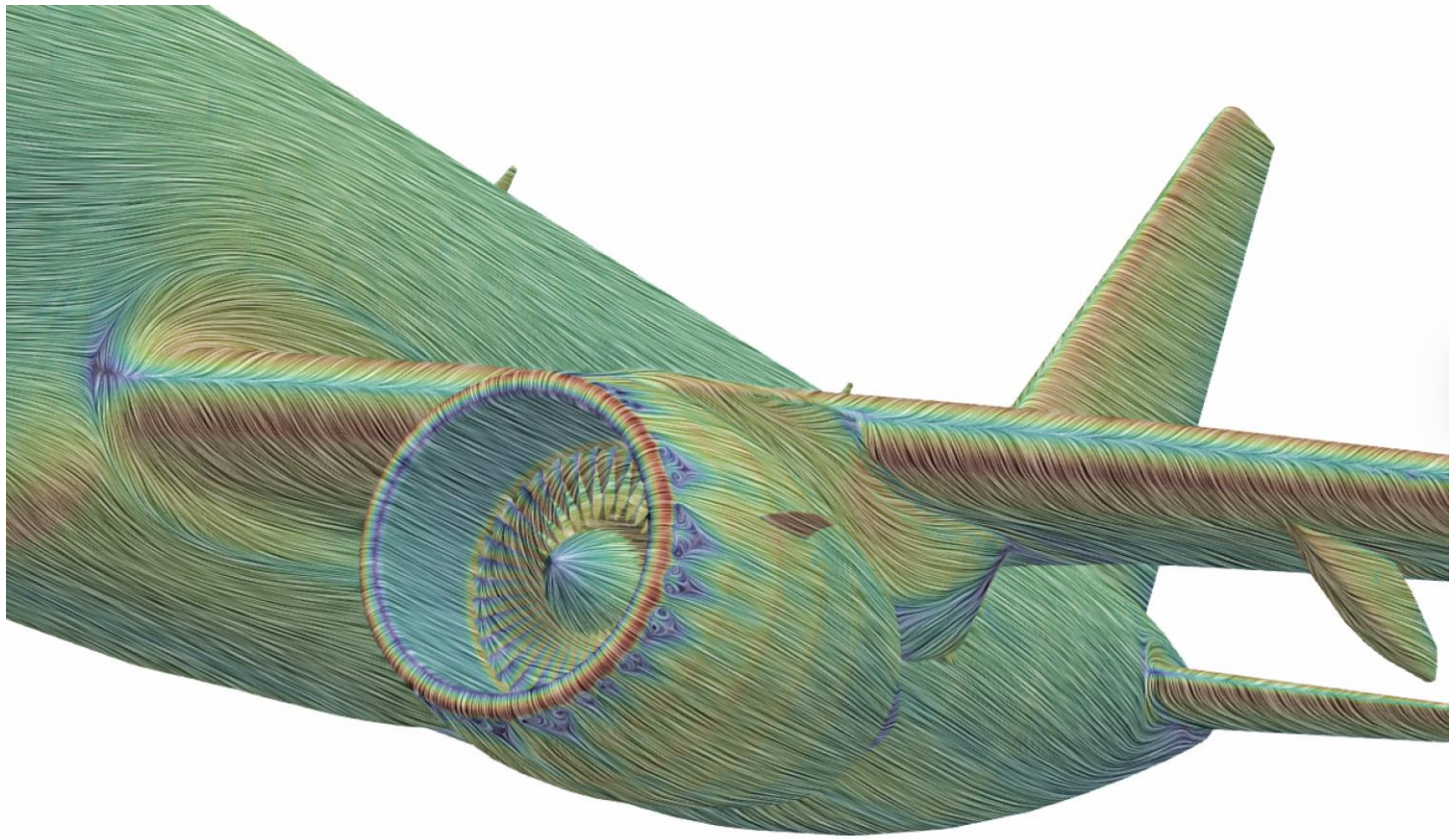
Matej Forman

OpenFOAM training

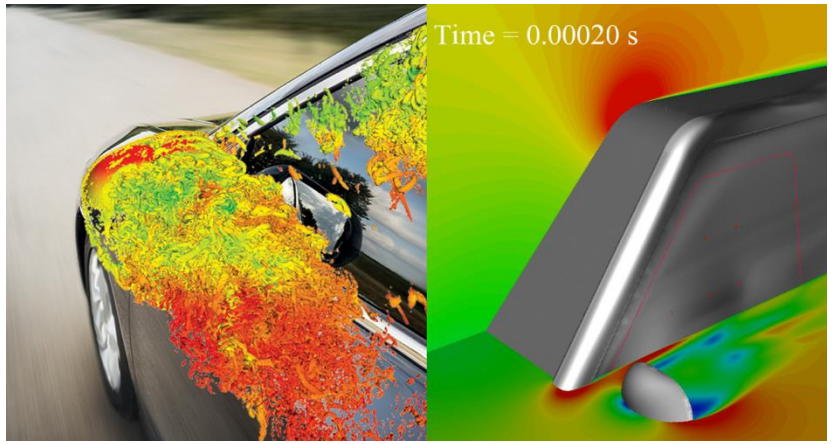
Content

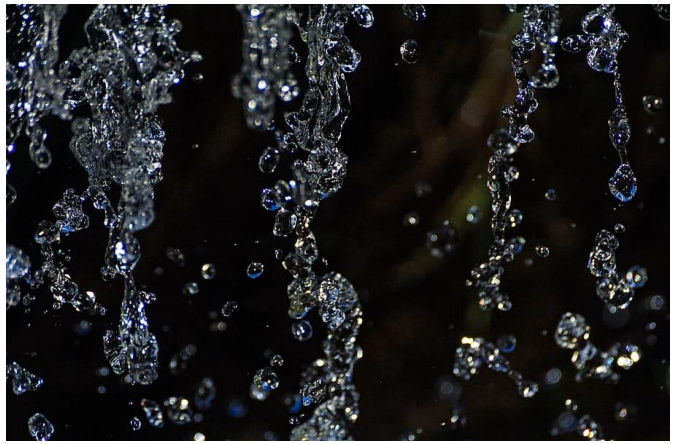
1. Introduction
2. OpenFOAM setup structure
3. Workflow and validation
4. Combustion case
5. Conical Diffusor case

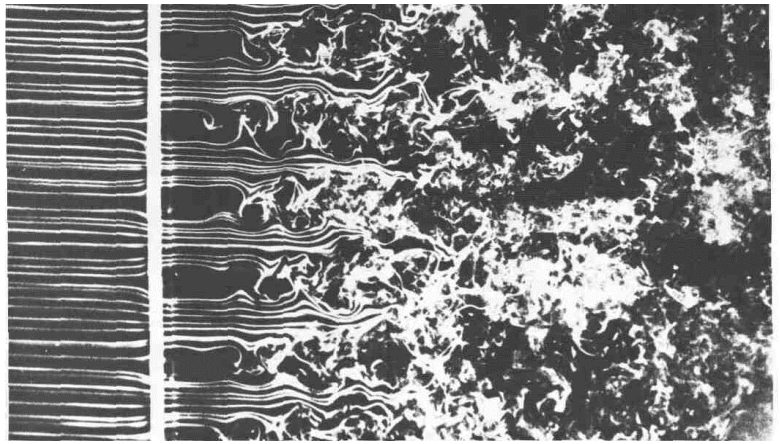
6. Documentation
7. GPU with 2606 version



What type of fluid phenomena is typically modelled?



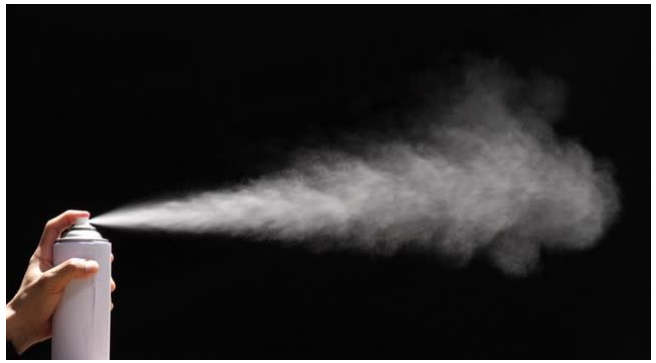














Introduction

OpenFOAM

- Free, open source software, licensed under the GNU General Public Licence
- Produced by Keysight Technologies Inc.,
- Released every 6 month by Keysight Technologies Inc.
- Distributed as source code; pre-compiled in Docker container for Linux, MS Windows and macOS (see `www.openfoam.com`)
- For computational fluid dynamics (CFD) (and other continuum mechanics)
- For real, 3-dimensional problems in science/engineering
- Designed as a programmable toolbox: “Matlab for CFD”
- Top level code represents the equations being solved, *e.g.*

$$\frac{\partial \rho \mathbf{U}}{\partial t} + \nabla \bullet \rho \mathbf{U}\mathbf{U} - \nabla \bullet \rho \mathbf{R} = -\nabla p$$

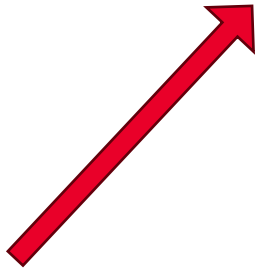
```
solve
(
    fvm::ddt(rho, U)
  + fvm::div(phi, U)
  + turbulence->divDevRhoReff(U)
  ==
  - fvc::grad(p)
);
```

Introduction

What is CFD?

? Colours For Directors
Codes From Diletants ?

Sometimes it is.
Most of the time it is an engineering tool
to resolve problems we meet.
Most typically the colourful pictures
Are really only for directors. Engineers are looking
For graphs, and average values, deltas ….



Computational Fluid Dynamics

A tool to describe a fluid flow problem
Using numerical methods to solve
Navier-Stokes equations – PDE
describing the flow. These equations
are quite often accompanied by
others describing physical processes
like heat transfer, combustion,
thermal radiations and others.

Introduction

Solution of Navier-stokes equations

Transport equation is a second order PDE describing the conservation of the given quantity as φ

$$\underbrace{\frac{\partial \rho \Phi}{\partial t}} + \underbrace{\nabla \cdot (\rho \mathbf{U} \Phi)} - \underbrace{\nabla \cdot (\rho \Gamma_\Phi \nabla \Phi)} = \underbrace{S_\Phi(\Phi)}$$

Rate of increase of φ In fluid element

Net rate of flow of φ out of element

Rate of increase due to diffusion

Rate of increase Due to the sources

- Any quantity could be φ
- When it is U, the equation is a momentum equation, when it is h (enthalpy) it is an thermal energy equation, when it is T, it is a temperature transport equation

Introduction

Solution of Navier-stokes equations

$$\frac{\partial \rho \Phi}{\partial t} + \nabla \cdot (\rho \mathbf{U} \Phi) - \nabla \cdot (\rho \Gamma_{\Phi} \nabla \Phi) = S_{\Phi}(\Phi)$$

Momentum equation

$$\frac{\partial \rho(U)}{\partial t} + \nabla \cdot (\rho \mathbf{U}\mathbf{U}) - \nabla \cdot (\rho \nu \nabla \mathbf{U}) = S_{\mathbf{U}}(\mathbf{U})$$

Mass conservation equation

$$\frac{D\rho}{Dt} + \rho (\nabla \cdot \mathbf{V}) = 0$$

Can we solve the system numerically?

Can we describe the system of equation we can solve with algebraic solvers?

$$A\mathbf{x} = \mathbf{B}$$

Introduction

Finite volume discretisation

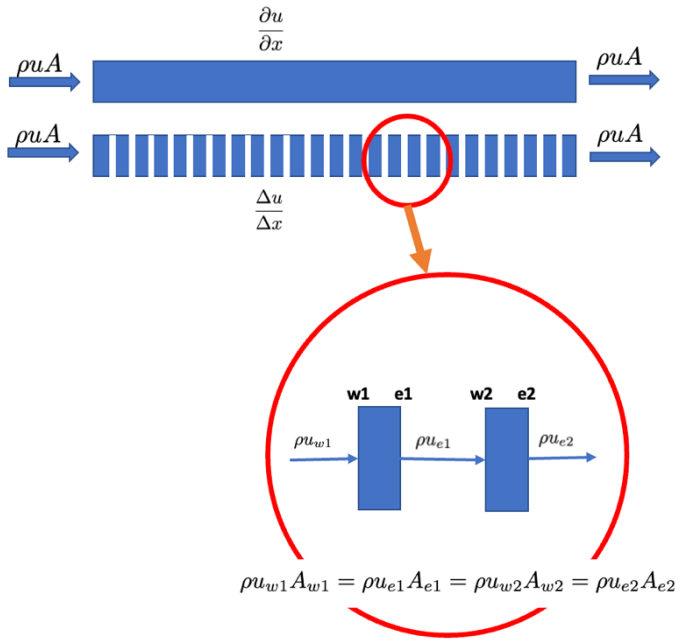
$$\frac{\partial \rho \Phi}{\partial t} + \nabla \cdot (\rho \mathbf{U} \Phi) - \nabla \cdot (\rho \Gamma_\Phi \nabla \Phi) = S_\Phi(\Phi) \longrightarrow A\mathbf{x} = \mathbf{B}$$

Conditions:

- Discretised volumes must be larger then the mean free path of molecules by orders of magnitude
- The discretised volume is in thermal equilibrium
- Let us concentrate on the finite volume method

- Value in the cell volume is constant
- Smooth profile is replaced by descrete values
- Precision grows with density of the mesh
- Main part of the correct linearisation of the problem
  is estimating values on the faces (w1, e1 …)
- Second part is the choice of the linear solver

- OpenFOAM stores values at the cell centres
- We are conserving fluxes at the faces



Introduction

Finite volume discretisation

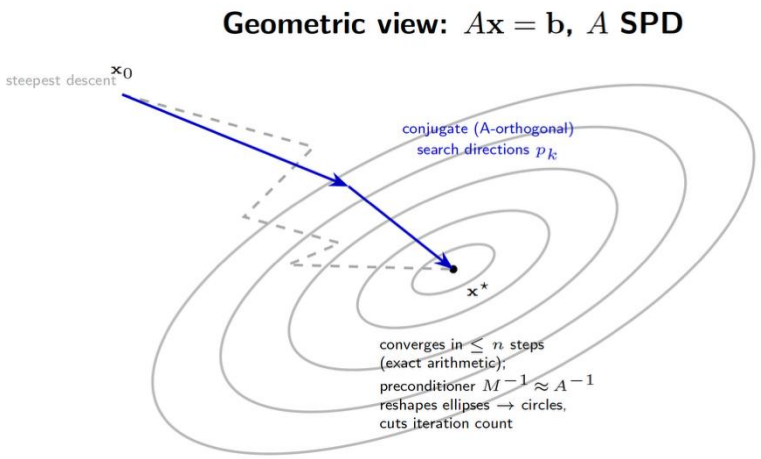
$$\frac{\partial \rho \Phi}{\partial t} + \nabla \cdot (\rho \mathbf{U} \Phi) - \nabla \cdot (\rho \Gamma_\Phi \nabla \Phi) = S_\Phi(\Phi) \longrightarrow A\mathbf{x} = \mathbf{B}$$

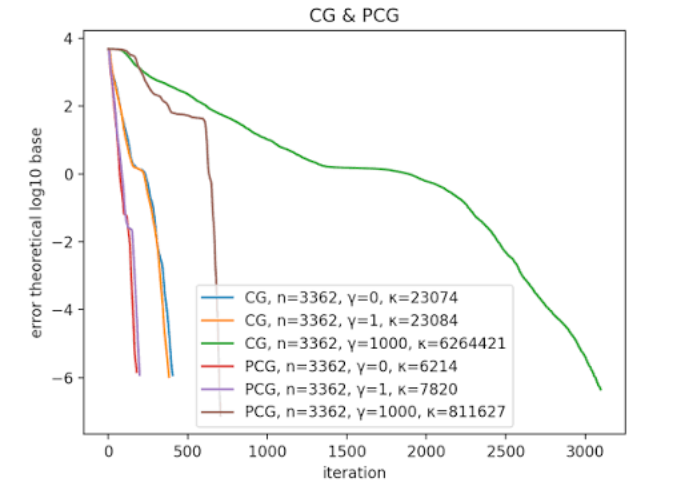
OpenFOAM use iterative solvers:

Conjugate gradient type
and multigrid

$$\begin{bmatrix} 300 & -100 & 0 & 0 & 0 \\ -100 & 200 & -100 & 0 & 0 \\ 0 & -100 & 200 & -100 & 0 \\ 0 & 0 & -100 & 200 & -100 \\ 0 & 0 & 0 & -100 & 300 \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \\ T_5 \end{bmatrix} = \begin{bmatrix} 200T_A \\ 0 \\ 0 \\ 0 \\ 200T_B \end{bmatrix}$$

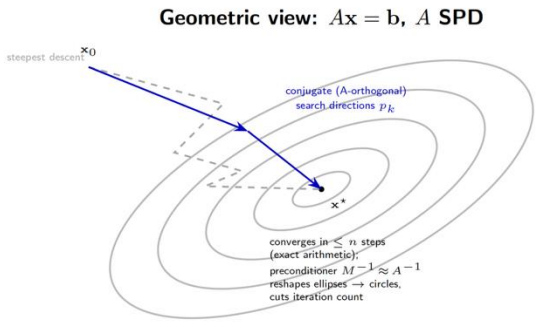
Iterative solution





Introduction

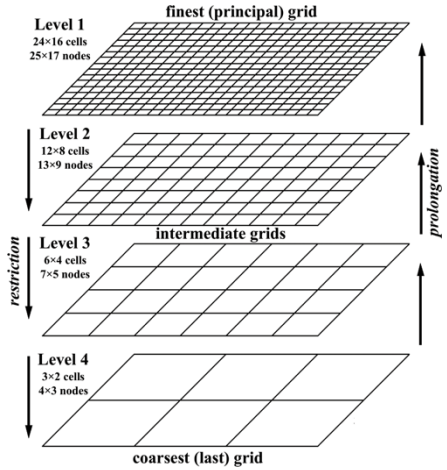
Iterative solvers



In OpenFOAM we use:

PCG – pre-conditioned CG solver for symmetric matrixes (pressure correction equation and other Laplacian equations)

GAMG – geometric algebraic multigrid for pressure equation



PBiCGStab – for assymetric matrixes (momentum and other transport equations)

smoothSolver - for simple flows faster than PBiCGStab, but not very robust.

Introduction

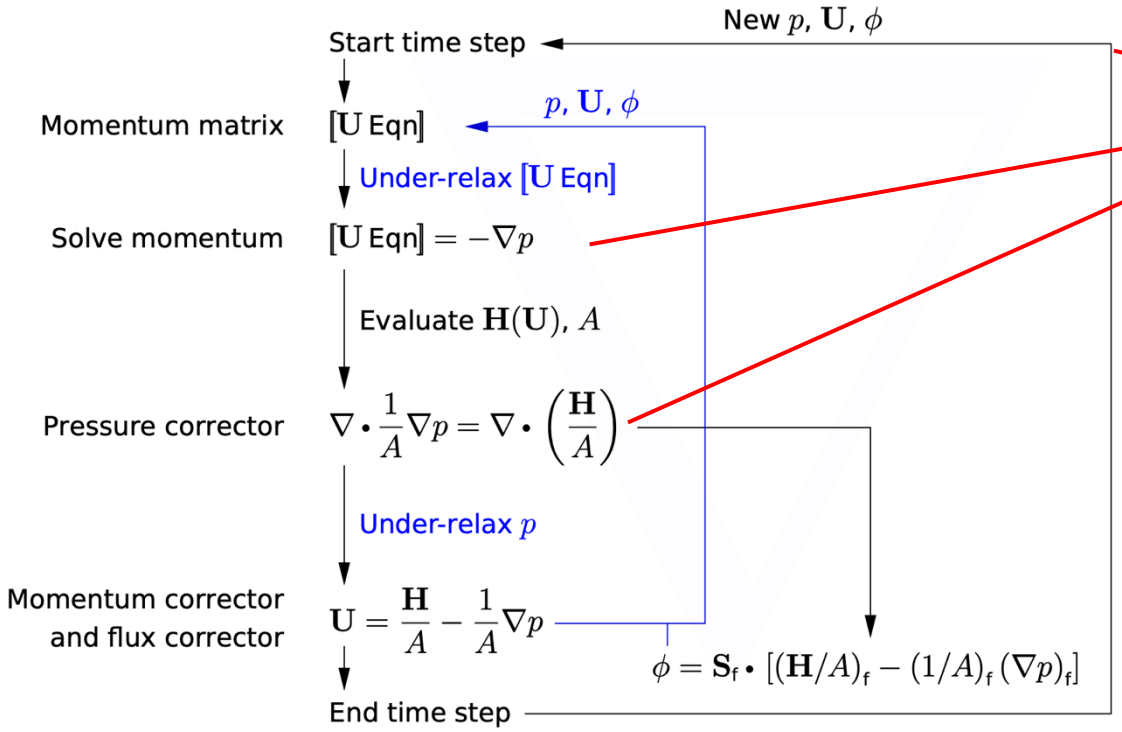
Pressure velocity coupling

- OpenFOAM solves for equation one by one
- Coupling algorithm provides coupling

SIMPLE – used for steady state simulations

PIMPLE – used for transient simulation

SIMPLE: Semi-implicit methods pressure-linked equations




```
Time = 1

smoothSolver:  Solving for Ux, Initial residual = 1,
smoothSolver:  Solving for Uy, Initial residual = 1,
GAMG:  Solving for p, Initial residual = 1, Final res
time step continuity errors : sum local = 1.19733, gl
smoothSolver:  Solving for epsilon, Initial residual
bounding epsilon, min: -1.98669 max: 1080.25 average:
smoothSolver:  Solving for k, Initial residual = 1, F
ExecutionTime = 0.35 s  ClockTime = 0 s

Time = 2

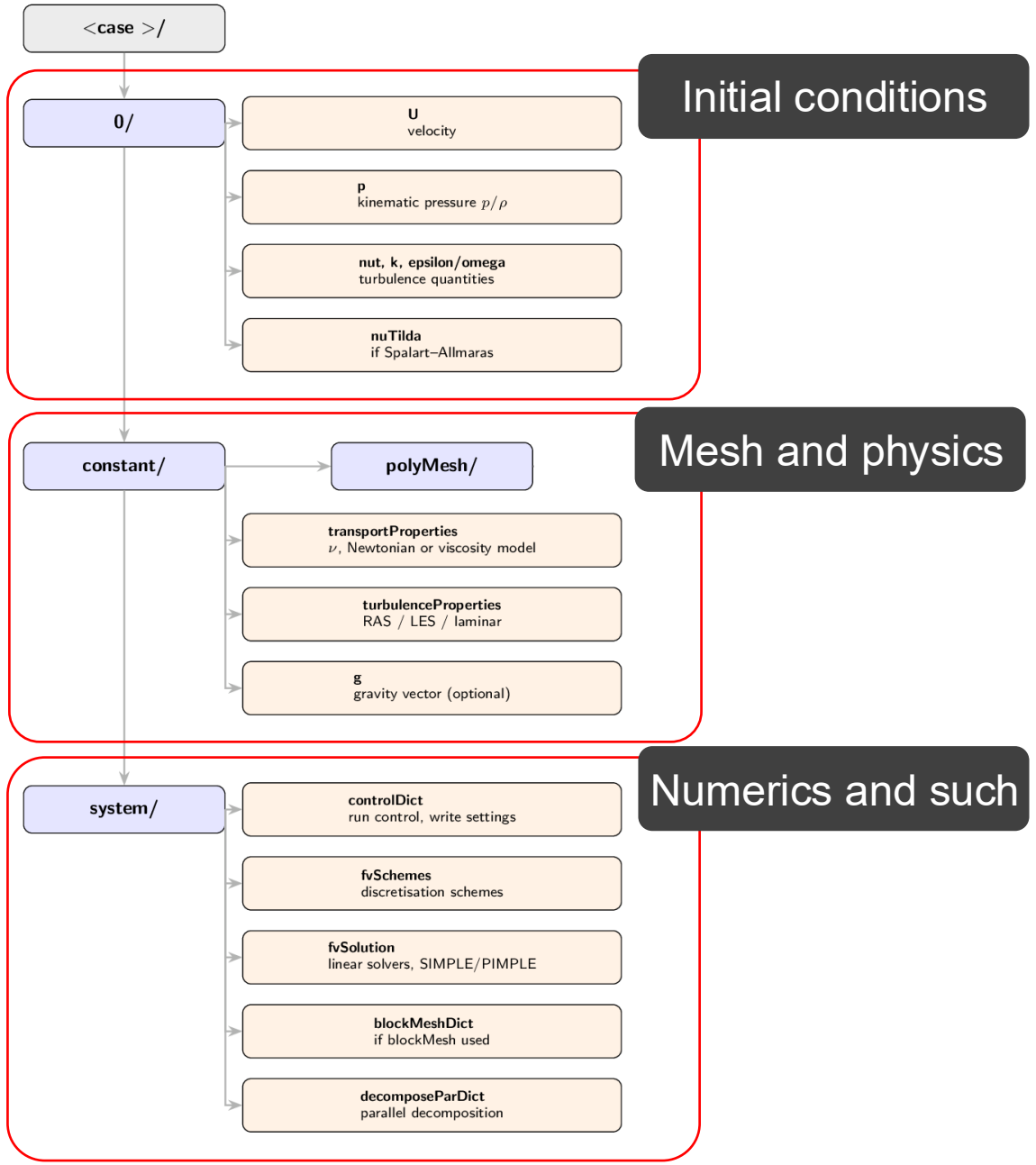
smoothSolver:  Solving for Ux, Initial residual = 0.4
smoothSolver:  Solving for Uy, Initial residual = 0.2
GAMG:  Solving for p, Initial residual = 0.052242, Fi
time step continuity errors : sum local = 4.02676, gl
smoothSolver:  Solving for epsilon, Initial residual
bounding epsilon, min: -40.6096 max: 23853.2 average:
smoothSolver:  Solving for k, Initial residual = 0.41
ExecutionTime = 0.4 s  ClockTime = 0 s
```

Introduction

OpenFOAM case structure

- OpenOAM is set using a series of text files
- Such structure allows easy scripting

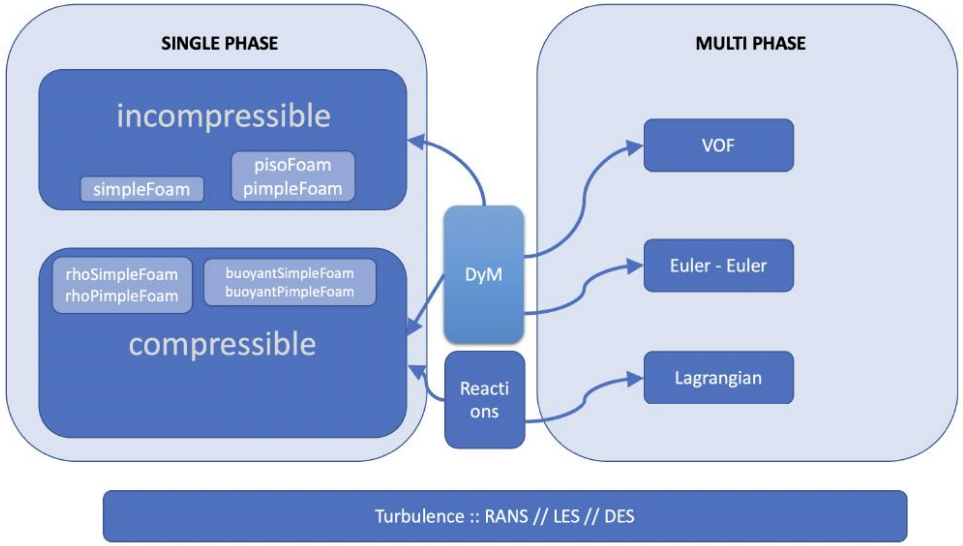
- C+ style free format files
- It is read from top to bottom
- OpenFOAM is using search function to read



Introduction

Basic structure of the applications

OpenFOAM is not compiled into one monolitic application but rather distributed into many smaller applications we call solvers and utilities. These applications are organised into a hiearchy following the physical phenomena to solve.



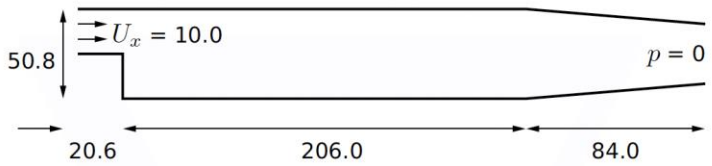
Logic of the solver naming and application distribution:

- PISO,PIMPLE solvers for transient flows
- SIMPLE solvers for steady state flows
- different solver = different physics or solution approach
- Only turbulence, radiation and DyM can he hot–plugged with a list of solvers
- Setup guidance provided via tutorials

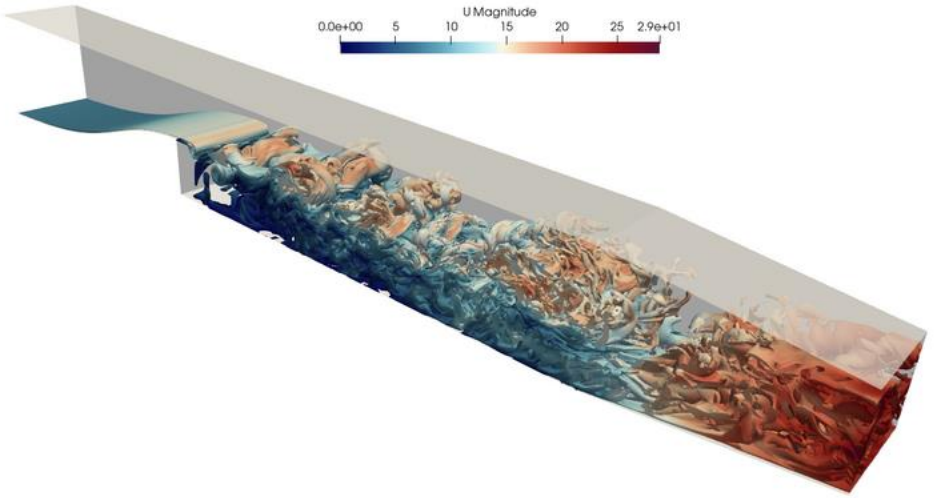
Introduction

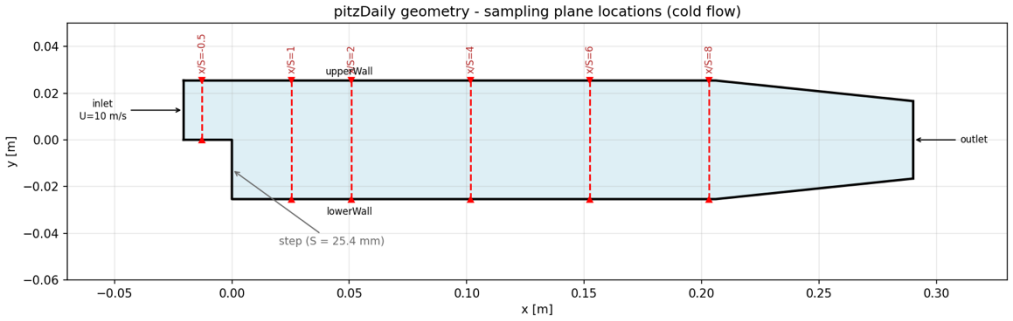
First run - pitzDaily

Backward-facing step (Pitz-Daily) case



- Steady, incompressible flow
- Inlet (left) with fixed velocity U_x = 10 m/s
- Outlet (right) with fixed pressure p = 0 m^2/s^2
- Pressure unit for incompressible flows is defined as Pa/ρ which simplifies to m^2/s^2
- No-slip walls ($\mathbf{U} = \mathbf{0}$)
- Kinematic viscosity (air) $\nu = \mu/\rho$ = 1.0 × 10^{-5}
- Turbulence model: $k - \epsilon$;





Running simple case

In directory pitzDailyColdFlow run:

```
$ blockMesh >log.blockMesh
$ simpleFoam >log.simplefoam
```

Or run:

```
$ ./Allrun
```

Allrun

```
#!/bin/sh

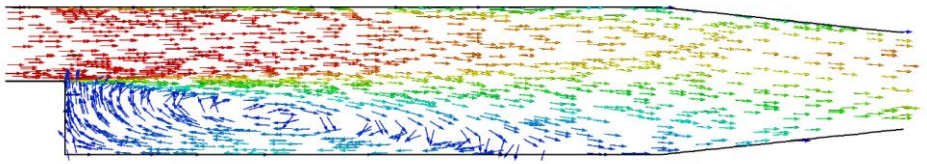
blockMesh >log.blkMesh
renumberMesh -overview >log.renumberMesh
simpleFoam >log.simpleFoam

# postprocessing
./postProcessingScripts/plotGeometrySketch.py
```

- OpenFOAM sends output to the terminal and can be redirected standard Linux ways.
- Successful execution writes End to the output.

- Results are stored in time-directories (100, 200 …)
- Postprocessing is done with ParaView

- Probed data from the flow are stored in postProcessing directory



Running simple case

pitzDaily: running the case - initialising

```
/*---------------------------------------------------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  v2506                                 |
|   \\  /    A nd           | Web:      www.OpenFOAM.com                      |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
Build  : v2506
Arch   : "LSB;label=32;scalar=64"
Exec   : simpleFoam
Date   : Feb 27 2025
Time   : 13:18:22
Host   : "ed9bca5d66bb"
PID    : 783
I/O    : uncollated
Case   : /home/ofuser/workingDir/OpenFOAM/forman-v2506/run/upgradeMaterials/pitzDaily
nProcs : 1
trapFpe: Floating point exception trapping enabled (FOAM_SIGFPE).
fileModificationChecking : Monitoring run-time modified files using timeStampMaster (fileModificationSkew 10)
allowSystemOperations : Allowing user-supplied system call operations

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //
Create time

Create mesh for time = 0


SIMPLE: convergence criteria
    field p       tolerance 0.01
    field U       tolerance 0.001
    field "(k|epsilon|omega|f|v2)"       tolerance 0.001

Reading field p

Reading field U

Reading/calculating face flux field phi
```

Running simple case

pitzDaily: screen output

```
Time = 2
smoothSolver: Solving for Ux, Initial residual = 0.449919, Final residual = 0.0314485,No Iterations 5
smoothSolver: Solving for Uy, Initial residual = 0.276409, Final residual = 0.0253638, No Iterations 4
```

- smoothSolver is the chosen linear-solver for the U equation
- Linear-solver decouples vector equation for U into components Ux and Uy
- Iterates until Final residual $<$ relTol $\times$ Initial residual
- fvSolution::solvers (solvers subdictionary of system/fvSolution):

```
34 "(U|k|epsilon|omega|f|v2)"
35 {
36         solver          smoothSolver;
37         smoother        symGaussSeidel;
38         tolerance       1e-05;
39         relTol          0.1;
40 }
```

Running simple case

pitzDaily: screen output (2)

```
GAMG: Solving for p, Initial residual = 0.0507773, Final residual = 0.00479054, No Iterations 16
time step continuity errors : sum local = 4.34847, global = 0.30184,cumulative = -0.0167273
```

- GAMG is the chosen linear-solver for the p equation
- p equation solved 1 time because nNonOrthogonalCorrectors = 0
- Errors in mass continuity presented as:
  - sum local: sum of magnitudes of continuity errors in each cell
  - global: across the boundary of the domain
  - cumulative: global error accumulated over time
- fvSolution::solvers

```
20 p
21 {
22     solver          GAMG;
23     tolerance       1e-06;
24     relTol          0.1;
25     smoother        GaussSeidel;
26 }
```

Running simple case

fvSolution

```
solvers
{
    p
    {
        solver          GAMG;
        tolerance       1e-06;
        relTol          0.1;
        smoother        GaussSeidel;
    }

    "(U|k|epsilon|omega|f|v2)"
    {
        solver          smoothSolver;
        smoother        symGaussSeidel;
        tolerance       1e-05;
        relTol          0.1;
    }
}

SIMPLE
{
    nNonOrthogonalCorrectors 0;
    consistent      yes;

    residualControl
    {
        p               1e-2;
        U               1e-3;
        "(k|epsilon|omega|f|v2)" 1e-3;
    }
}

relaxationFactors
{
    equations
    {
        U               0.9; // 0.9 is more stable but 0.95 more convergent
        ".*"            0.9; // 0.9 is more stable but 0.95 more convergent
    }
}
```

Algebraic solvers

P-U alg.

Running simple case

fvSchemes

```
ddtSchemes  {    default          steadyState; }
gradSchemes {    default          Gauss linear;}
divSchemes
{
    default          none;
    div(phi,U)       bounded Gauss linearUpwind grad(U);
    turbulence       bounded Gauss limitedLinear 1;
    div(phi,k)       $turbulence;
    div(phi,epsilon) $turbulence;
    div(phi,omega)   $turbulence;
    div(nonlinearStress) Gauss linear;
    div((nuEff*dev2(T(grad(U))))) Gauss linear;
}
laplacianSchemes { default          Gauss linear corrected;}

interpolationSchemes {    default          linear; }

snGradSchemes {  default          corrected; }

wallDist {    method           meshWave; }
```

- Discretisation and interpolation schemes have direct effect on stability and precision of the solution.
- Provided setup offers good balance
- Best practice is no touch this file

Running simple case

controlDict

- Main file in system directory
- Controls when to start, finish
- User register here the postprocessing functions

```
application     simpleFoam;
startFrom       startTime;
startTime       0;
stopAt          endTime;
endTime         2000;
deltaT          1;
writeControl    timeStep;
writeInterval   100;
purgeWrite      0;
writeFormat     ascii;
writePrecision  6;
writeCompression off;
timeFormat      general;
timePrecision   6;
runTimeModifiable true;
functions
{
    #includeFunc streamlines
    #include "sampleDict"
}
```

Explicit number of solution steps.
Keep the same for fair comparison

How often to write data
To reduce IO set it to $endTime

Binary or ascii result format

Register additional post-processing

Simple case

Convergence - How do I recognise whether the case run successfully or diverged?

- Check the output log file.

Check Initial residuals – should be below 1e-4

```
Time = 281

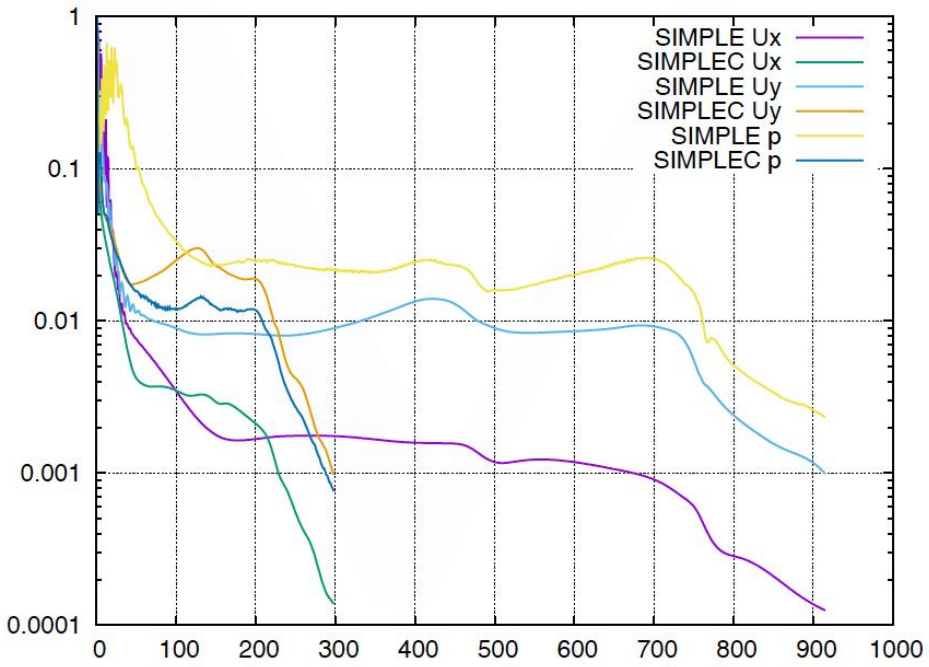
smoothSolver:  Solving for Ux, Initial residual = 0.000121518, Final residual = 1.1817e-05, No Iterations 5
smoothSolver:  Solving for Uy, Initial residual = 0.00098379, Final residual = 6.86399e-05, No Iterations 6
GAMG:  Solving for p, Initial residual = 0.000776765, Final residual = 7.0102e-05, No Iterations 5
time step continuity errors : sum local = 0.00334784, global = -0.000238938, cumulative = 1.13829
smoothSolver:  Solving for epsilon, Initial residual = 0.000120658, Final residual = 6.89313e-06, No Iterations 3
smoothSolver:  Solving for k, Initial residual = 0.000223528, Final residual = 1.40393e-05, No Iterations 4
ExecutionTime = 3.31 s  ClockTime = 4 s
```

Check mass conservation error:

```
Time = 281

smoothSolver:  Solving for Ux, Initial residual = 0.000121518, Final residual = 1.1817e-05, No Iterations 5
smoothSolver:  Solving for Uy, Initial residual = 0.00098379, Final residual = 6.86399e-05, No Iterations 6
GAMG:  Solving for p, Initial residual = 0.000776765, Final residual = 7.0102e-05, No Iterations 5
time step continuity errors : sum local = 0.00334784, global = -0.000238938, cumulative = 1.13829
smoothSolver:  Solving for epsilon, Initial residual = 0.000120658, Final residual = 6.89313e-06, No Iterations 3
smoothSolver:  Solving for k, Initial residual = 0.000223528, Final residual = 1.40393e-05, No Iterations 4
ExecutionTime = 3.31 s  ClockTime = 4 s
```

Initial residual plot



Running case in parallel

Domain decomposition splits the mesh (and fields) into numberOfSubdomains pieces for parallel execution (decomposePar + mpirun -np N <solver> -parallel). The method is set via method in system/decomposeParDict. Goal: minimize processor-boundary ("halo") faces while balancing cell count per domain, since inter-processor communication is the main parallel-scaling bottleneck.

Decomposition methods:

- **Hierarchical -** decomposes sequentially along a specified coordinate order (order keyword, e.g. xyz), refining one direction at a time. Purely geometric — no awareness of connectivity.
- **Scotch (and ptscotch) -** reats the mesh as a dual graph (cells = graph nodes, faces = graph edges) and partitions it to minimize edge cuts (processor-boundary faces) while balancing cell count. Ptscotch is the MPI-parallel variant of the same algorithm. Good load balancing across arbitrary N. Scotch may fail for 400 and more codes.
- **Metis and KaHip** – other automatic mesh decomposition methods.

- Except mesh generator blockMesh, all OpenFOAM applications work in parallel

Running case in parallel

system/decomposeParDict:

```
numberOfSubdomains 10;

method scotch;
```

Run:

$ decomposePar

-dry-run Test without writing

-verbose Additional verbosity



```
Calculating distribution of cells. nCells = 43095
Decomposition method scotch [8] (region region0)

Finished decomposition into 8 domains in 0.55 s

Decomposition details with (proc faces) for each processor connection

Part[0] cells:5434 neigh:4 faces:819  (1 463) (3 148) (4 91) (5 117)
Part[1] cells:5343 neigh:4 faces:809  (0 463) (2 182) (3 8) (5 156)
Part[2] cells:5415 neigh:2 faces:648  (1 182) (3 466)
Part[3] cells:5401 neigh:3 faces:622  (0 148) (1 8) (2 466)
Part[4] cells:5408 neigh:3 faces:689  (0 91) (5 390) (6 208)
Part[5] cells:5343 neigh:4 faces:819  (0 117) (1 156) (4 390) (6 156)
Part[6] cells:5418 neigh:3 faces:551  (4 208) (5 156) (7 187)
Part[7] cells:5333 neigh:1 faces:187  (6 187)

Cells min:5333 max:5434 median:5408 (100.481%)
Neigh min:1 max:4 median:3 (133.333%)
Faces min:187 max:819 median:689 (118.868%)
```

To get the detailed statistics about the decomposition without writing decomposed files

More details can be found in: $FOAM_ETC/caseDicts/annotated/decomposeParDict

Compiler

- Compilation options are set in $WM_PROJECT_DIR/etc/prefs.sh which overwrites the $WM_PROJECT_DIR/etc/bashrc

  Individual settings for libraries are set in $WM_PROJECT_DIR/config.sh/

Compiler and mpi

Compilers

Family	Notes
Gcc	GNU g++ (default on Linux)
Clang	LLVM clang++ (default on macOS via WM_COMPILER_TYPE=system, and available on Linux)
Icc	Intel classic compiler (legacy, deprecated upstream)
Icx	Intel oneAPI (icpx) — current Intel toolchain
Nvidia	NVIDIA HPC SDK (nvc++)
Cray	Cray compiler wrappers (CC)
Arm	Arm Compiler for HPC (ARM64 only)
Fujitsu	Fujitsu compiler (ARM64, e.g. Fugaku)
Mingw	Windows cross-compile (Linux host)

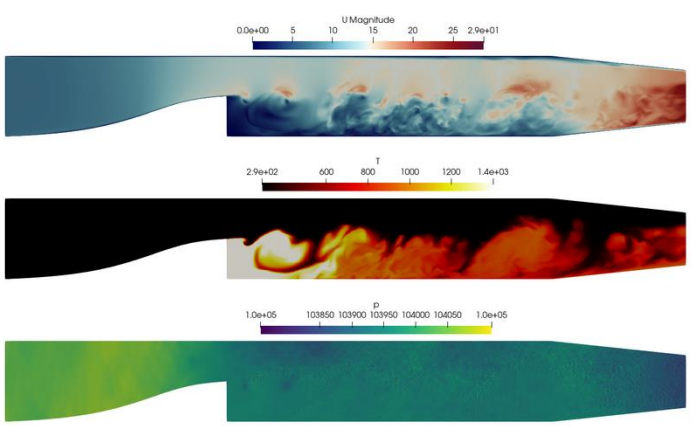
MPI libraries

WM_MPLIB **value**	**Library**
SYSTEMOPENMPI (+[1-9]/-N variants)	System-installed Open MPI (your current setup, via Homebrew)
OPENMPI	ThirdParty-built Open MPI
SYSTEMMPI	Generic system MPI (compiler-wrapper based)
USERMPI	User-defined MPI, manually configured paths
EASYBUILDMPI	EasyBuild-provided MPI module
MPICH	MPICH (ThirdParty or system)
MPICH-GM	MPICH over Myrinet GM (legacy)
MVAPICH2	MVAPICH2 (InfiniBand)
INTELMPI	Intel MPI
MSMPI	Microsoft MPI (Windows)
FJMPI	Fujitsu MPI
CRAY-MPICH	Cray's MPICH
HPMPI	HP-MPI (legacy)
QSMPI	QsNet MPI (legacy)
SGIMPI	SGI MPT

Validation cases overview

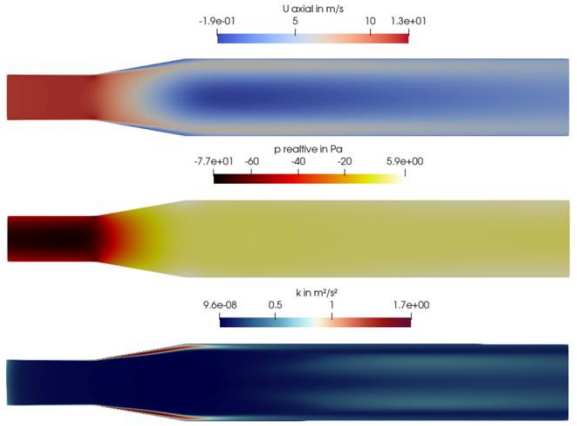
- Cases are taken and adopted from the list of HPC technical committee of OpenFOAM governance. [https://wiki.openfoam.com/High_Performance_Computing_(HPC)_Technical_Committee]
- https://develop.openfoam.com/committees/hpc

Pitz&Daily Combustor



- Many files (for each combustion specie)
- Small and easy-to-scale structured mesh
- Well validated

Conical diffuser



- Smaller number of equations to solve
- Small and easy-to-scale structured mesh
- Well validated

Validation cases overview

- Both cases have their run scripts executing case in parallel and creating validation graphs
- Both cases have caseParam file where user may vary mesh size with a single parameter meshScale and also parallelisation:

```
# Parallel decomposition
# nProcs = nNodes × nSocketsPerNode × nCoresPerSocket
# multiLevel scotch decomposition is used automatically.
nNodes=1
nSocketsPerNode=1
nCoresPerSocket=10
```

Allpost script creates HTML report with validation graphs and also report with overall relative error across all velocity profiles.

- Scripts:
  - Allsetup
  - Allpre
  - Allrun
  - Allpost

  } Allall

Validation cases overview

Expected level of error ~ 10 – 20 %

Quantity	Upstream (z=-25→60mm)	In/after diffuser (z≥100mm)	Why
U (axial)	NRMSE ~5–15%	~10–25%, worse near separation (z≈250–330mm)	Mean-flow deceleration well captured until adverse-pressure-gradient separation onset
W (swirl)	NRMSE ~10–20%	~15–30%	Swirl decay rate under-predicted by linear eddy-viscosity closures lacking streamline-curvature/rotation correction
k	NRMSE ~20–40%	**often 50–100%+**	Linear EVMs (incl. SST) systematically mis-predict turbulence anisotropy under swirl+curvature — classic known failure mode for this case

OpenFOAM performance

- Workflow consist of many steps
- Wall-clock time vs ExecutionTime/ClockTime from log are the basic parameters
- Parallel performance tends to be good, when the load is distributed evenly and single core has domain size in the range from 50 000 – 100 000 of cells
- OpenFOAM was heavily tested and improved for exa-size cases (~1 billion cells) see exafoam.eu for details
- IO opearations can be minimised with collating output field files.
- OpenFOAM can also work with ADIOS to optimise IO operations

https://www.openfoam.com/news/main-news/openfoam-v1612/community#community-parallel-io

system/controlDict:

```
OptimisationSwitches
{
    fileHandler collated;
}
```

OpenFOAM performance

- Global defaults: `$WM_PROJECT_DIR/etc/controlDict` (system-wide, do not edit)
- Case override: `system/controlDict` → `OptimisationSwitches { ... }` sub-dict
- Runtime override: command-line flag or environment variable (highest priority)

Precedence order (highest wins)
- Command line (`-fileHandler`, `-D<switch>=value` where supported)
- Environment variable (`FOAM_<SWITCH>`)
- Case `system/controlDict`
- User `~/.OpenFOAM/<version>/controlDict`
- Global `etc/controlDict`

Common switches

system/controlDict:

```
OptimisationSwitches
{
    fileHandler             collated;   // I/O backend: uncollated | collated | masterUncollated
    maxThreadFileBufferSize  1e9;       // bytes; enables threaded background writes
    commsType               nonBlocking; // scheduled | blocking | nonBlocking
    floatTransfer           0;          // 1 = single precision comms (speed vs accuracy)
    nProcsSimpleSum         0;          // reduce-op tuning for large core counts
}
```

Latest news - GPU

The first release to support GPU offloading

- **First OpenFOAM release with GPU support**
  - built on the C++17/20 std::execution policy — loops parallelize automatically across GPU or shared-memory CPU cores,
  - no new user API.
- **Key enablers:**
  - cell-based (not face-based) algorithms to avoid race conditions;
  - memory allocated where needed via a Umpire-backed memory pool;
  - expression-templated on-the-fly interpolation to avoid intermediate field allocation.
- **Joint effort:**
  - Keysight/OpenCFD + STFC + Exeter University + AMD/Nvidia (hardware + engineering);
- **User-facing API** (field algebra, linear solvers, BC evaluation) is unchanged — this is a low-level rewrite.

See release notes: https://www.openfoam.com/news/main-news/openfoam-v2606/infrastructure#gpu

Where to find reliable information:

- Openfoam.com
- https://gitlab.com/openfoam/core/openfoam
- exafoam.eu